Witness-CL: Evidence-Preserving Online Learning and the Cost of Learning from Probes

Samuel Mausberg
AI-assisted research draft; author review and independent replication pending

Abstract

We study when experience warrants a change in future behavior under an explicit retention contract. Witness-CL learns a finite class of hidden transducers from executed reset traces and admits immutable programs only after exact baseline-relative checking. A greedy decision-regret probe rule fails on complementary information. We implement a bounded two-probe successor, preserve the failed rule, and compare stronger cheap controls. All new selectors solve a four-world two-bit construction at zero loss, while three-bit parity at B0 defeats depth two and is solved by the cheap informative rule. In a frozen 40-stream, 4,800-episode study, depth two improves mean reward by 0.3938 over free information seeking (95% paired interval $[0.1585, 0.6290]$), but loses 0.1948 to bounded optimism and uses $8.88\times$ the cheap control's planning time. Independent replay finds no protected-value, evidence-cover, or prepaid-budget violations under the finite contract. Lean 4.19.0 checks 63 statements, including an executable interpreter/filter bridge; exhaustive small-family tests compare 262,144 filter memberships with Python. The v6 study freezes all neural weights; prior local-model pilots remain separate. These results support a narrow information-acquisition mechanism while rejecting its efficiency superiority. They do not establish learned abstraction validity, general no-forgetting, alignment, or a native continual-learning benchmark win.

1 Problem and the actual advance

CL-Bench evaluates whether frontier language agents improve across episodes that share learnable latent structure. Its strongest aggregate system uses raw-history in-context learning (ICL); this does not imply that every memory method loses on every metric or backbone. The paper reports immediate-observation overfitting and weak reuse across instances [1]. AgentCL uses controlled compositional, naive, and held-out streams to distinguish reuse from mere persistence; irrelevant memory can degrade performance [2]. These results motivate a more exact question: what part of experience should a learner treat as state, and what evidence warrants letting that state change its future behavior?

The open target is a cost-effective, deployed learner that improves across useful stateful tasks from its own episodes without an offline retraining phase and without losing valuable prior behavior. Merely preserving weights, transcripts, or a skill file does not imply preserving downstream competence. Conversely, not every episode can be required to improve if useful information sometimes requires a costly experiment. A meaningful solution must separate retained competence from exploration loss and account for both.

Inherited mechanism. The v0.3 ancestor of the supplied v0.4 bundle provides a finite list of possible worlds, observable complete state, continuation-aware improvement, incumbent retention, and prepaid exploration. Its synthetic late return matches rather than beats a strong full-history learner. Separate noisy-evidence and frozen-neural-module branches do not solve representation acquisition. We preserve that Git history and archive its paper and results rather than relabel them as new experiments.

Version 4. We replace the supplied world list and observed latent state with a learned set of hidden transducers consistent with executed traces. Only a structural class bound, reset semantics, finite alphabets, and public reward functions are supplied. The same loop now contains online neural proposal updates, exact model-set conditioning, immutable observation-contingent programs, paired continuation verification, and informative prepaid probes. No hidden world table, state, or regime label is given to the learner.

The advance is deliberately restricted. Discovering compatible latent models inside a correctly declared finite class is not discovering a sufficient abstraction of arbitrary language or computer-use traces. Our proposed high-impact direction is to learn affordable continuation contracts: compact predictive descriptions sufficient to justify useful behavioral changes. The reference makes this direction executable and falsifiable. Its current empirical result is not benchmark superiority.

Version 5 contribution and result. We audit the v0.4 bundle rather than assume its claims are verified. We compile and repair its abstract Lean proofs, test decision-directed exploration with a frozen rule, diagnose

its failure, execute local-model inference, and pin the native benchmark interface. The new exploration rule loses on the primary held-out metric. Its failed test and complementary-probe counterexample constrain the successor design; they are not presented as a solution to continual learning. The original v0.4 paper and data remain archived unchanged.

Version 6 contribution and result. The bounded successor gains reward over the zero-loss information rule but loses to bounded optimism while using more computation. Its three-bit failure and 30.7% UNKNOWN rate limit the usefulness of greater planning depth. The new executable Lean bridge narrows the gap between abstract evidence retention and actual interpreter semantics; a general Python refinement proof remains open. Section 11 gives the frozen protocol and measured results.

2 Prior work constrains the claim

Memory and executable experience. ACE develops evolving playbooks; ALMA meta-learns memory designs; EvoLib builds and consolidates reusable knowledge abstractions [3, 6, 12]. These are substantive adaptation mechanisms, not simply retrieval stores. ALMA's outer search also makes it important to distinguish prior design cost from strictly online adaptation. None of these comparisons has been reproduced here.

EvoTest evolves system configurations between episodes, while TTHE evolves the executable harness and commits changes using execution-derived proxy signals [4, 5]. TTHE's main evaluation adapts and scores on the same batch; its authors identify next-batch evaluation and stronger compute matching as open questions. A September 4, 2026 study constructs persistent versioned skills for computer-use agents, freezes the library during each iteration, and reports benefits over a configuration-matched empty-library control along with revision failures [13]. Versioned libraries and frozen execution snapshots are therefore not new contributions of Witness-CL. The unresolved distinction is when evidence supports a claim about later behavior, rather than only a favorable proxy or recent task.

Latent state and conservative learning. Predictive state representations already define state through predictions of future observable experiments [10]. Dynamic shielding learns an approximate automaton from black-box interaction and filters potentially unsafe exploration [11]. DeepSPI already combines online safe policy improvement, learned world models, and representations, with conditional theory and ALE-57 evaluation [8]. Its learned improvement mechanism is a direct antecedent; abstraction validity and the assumptions in its bounds remain substantive requirements. These are direct precedents, not peripheral related work. Our exact deterministic class is narrower than those applications.

Conservative exploration methods such as StepMix address explicit baseline performance constraints [9]. The prepaid debit argument here is not the first conservative-exploration theorem. Progressive networks already isolate previous learned components [7]; neither freezing old functions nor changing parameters only online is novel by itself. We claim an implemented combination with explicit failure tests, not established novelty of the combination or a new universal learning principle.

3 A precise latent-learning contract

Let A and O be known finite action and output alphabets. The unknown world $m^\star$ is a stationary deterministic transducer with at most K states, embedded in $S = \{0, \dots, K-1\}$. A complete table specifies

$$T_m : S \times A \longrightarrow S \times O. \tag{1}$$

Every episode begins at the same actual reset state 0. Hidden state labels and the table are never observed. The learner receives the executed trace $\tau = ((a_0, o_0), \dots, (a_{H-1}, o_{H-1}))$ at finite horizon H. There is no stochastic feedback in the integrated experiment.

A public objective g specifies a known bounded integer reward $r_g(o)$. A policy p is a complete immutable tree: $p_t : O^t \to A$. The reset return is

$$V_m(p, g) = \sum_{t=0}^{H-1} r_g(o_t), \qquad a_t = p_t(o_{<t}). \tag{2}$$

A protected incumbent $b_{n,g}$ is stored for each declared objective. The objective identifies the requested utility, not a hidden dynamics regime. The common initial anchor is $b_{0,g}$.

Let $\mathcal{D}_n$ contain all actually executed traces before episode n. Define

$$\mathcal{C}_n = \{m \in \mathcal{M}_K : \ \forall \tau \in \mathcal{D}_n, \ m \models \tau\}. \tag{3}$$

Here $m \models \tau$ means that the table realizes the observed trace. The model class has $(K|O|)^{K|A|}$ labelled complete tables. Supplying K is a substantial prior. Data consistency cannot prove that $m^\star \in \mathcal{M}_K$. Theorems below explicitly assume this coverage and unchanging reset semantics.

Four different claims. Preserving raw evidence is not the same as preserving a learned function. Preserving that function is not the same as preserving a routed incumbent's reset value. Preserving incumbent value does not require every exploratory episode to attain that value. We prove the third property under this contract

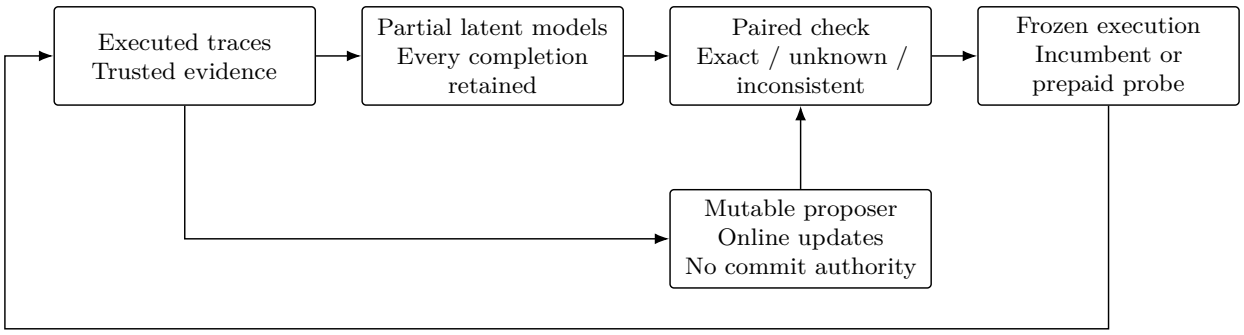

Figure 1: Implemented loop. Model rollouts do not become observed training labels. The proposer may forget internally; only checked immutable programs replace a protected incumbent. All guarantees are conditional on a correct finite class, stationary dynamics, declared rewards, and genuine resets.

and separately bound the fourth property's possible deficit. We do not prove that the mutable neural network itself avoids forgetting.

4 Learning an exact partial-model cover

A partial table c assigns some state–action edges and leaves the rest undefined. Its denotation is

$$[c] = \{m \in \mathcal{M}_K : \ c \subseteq T_m\}, \tag{4}$$

where inclusion means agreement on every assigned edge. A finite frontier $\mathcal{F}_n$ represents $\mathcal{C}_n = \bigcup_{c\in\mathcal{F}_n}[c]$. Initially the single all-undefined table represents the entire class. No complete list of worlds is passed to the learner.

To process one reset trace, pair every frontier table with state zero. For observed (a, o), retain a defined edge only when its output is o. For an undefined edge, branch over all K successor states, assigning output o in every branch. Retain the branch's hidden successor while processing the trace. Merge duplicate table–state pairs. At episode end discard the end state, since the next episode resets, and merge duplicate tables. Undefined edges are not filled with a likely default during inference.

Theorem 1 (Evidence-complete cover)**.** *With exact feedback and no search truncation, the update above represents exactly $\mathcal{C}_n$. If $m^\star \in \mathcal{M}_K$, every updated cover contains $m^\star$.*

Proof. Induct on observations within each reset trace. For any compatible complete table, a defined visited edge must have the observed output; an undefined edge has its true successor among the K branches. Thus no compatible table is lost. Conversely, every retained branch assigns the observed output to the visited edge and tracks its successor, so all its completions realize the processed prefix. Resetting the branch state between episodes is valid by assumption. Deduplication changes representation, not its union of completions. Induction over episodes gives equality; actual feedback gives truth membership. □

This cover can overlap and can grow exponentially. The implementation caps the frontier and marks incomplete inference rather than silently retaining only a likely subset. Once coverage is unknown, new certificates are not issued. An empty exact class is reported as INCONSISTENT, never as vacuous success.

Changing capacity. Appending a new candidate program does not change $\mathcal{C}_n$. Increasing K does. The implementation replays all raw evidence under the new bound, starts a new contract era from the anchor, and does not inherit old certificates or the old budget claim. This is an explicit online recovery operation, not evidence that a fitted smaller model was sound before expansion.

5 Checking a pair rather than trusting a model

For a candidate p and current incumbent b, define

$$L_n(p, b, g) = \min_{m\in\mathcal{C}_n} \big[V_m(p, g) - V_m(b, g)\big], \tag{5}$$

$$U_n(p, b, g) = \max_{m\in\mathcal{C}_n} \big[V_m(p, g) - V_m(b, g)\big]. \tag{6}$$

Computing these extrema separately by materializing every complete world is unnecessary when many transitions cannot affect the comparison.

The checker symbolically executes both programs in the *same* partial table. It expands an undefined edge only when one execution reads it, branching over its $K|O|$ possible assignments. Both executions share each assignment. It accumulates their reward difference and retains a complete witness for the minimum. Unvisited slots remain universally quantified.

Closing a suffix. At the same time index, if both executions have the same hidden state and identical remaining observation-contingent programs, their remain-

ing reward difference is zero for every completion. This follows by induction on future steps. Equal current observations alone do not justify closure: hidden states with the same emitted symbol can have different successors.

We compile remaining program trees into exact pair-local directed acyclic graph (DAG) IDs. The interning key is the complete tuple consisting of the current action and child IDs. Dictionary equality checks the whole tuple, so a hash collision cannot equate different programs. This is not a shapes-only cache or a learned equivalence prediction. DAG construction cost is included in timings.

Theorem 2 (Exact paired extrema)**.** *For a nonempty exact cover, a completed paired search returns precisely* (5) *and* (6)*. A returned lower-bound witness is a compatible complete table attaining the minimum.*

Proof. Expanding an unread edge partitions its completions into all possible assigned edges. A defined edge needs no split. Therefore every compatible world follows a search branch, and no branch introduces an incompatible visited edge. At the horizon the accumulated difference is constant over all unread completions. The same holds at an early closure because subsequent states, programs, observations, and rewards agree inductively. The minimum and maximum of these leaf differences are consequently the desired extrema. Any remaining undefined slots at a minimizing leaf may be completed arbitrarily, producing a compatible witness with the same difference. □

A node cap yields Unknown with no usable bound. It does not turn visited leaves into a confidence interval. For each partial, at most $u = \min(K|A|, 2H)$ distinct unresolved slots are read along a paired path, giving up to $(K|O|)^u$ assignments, in addition to horizon and frontier overhead. Full policy-tree construction itself costs $O(\sum_{t=0}^{H-1} |O|^t)$. Conditional sharing is not polynomial-time learning.

Corollary 3 (Maximality for the declared class)**.** *For a fixed candidate and incumbent, $L_n \geq 0$ is necessary and sufficient for nonnegative return difference in every world of $\mathcal{C}_n$. If $L_n < 0$, its witness refutes uniform admission within this contract.*

Proof. Sufficiency is the definition of a minimum. Necessity follows because the finite nonempty class attains that minimum. Theorem 2 supplies a witness. □

This does not make the overall learner optimal. A different experiment, candidate, objective, or weaker guarantee may yield higher return.

6 Online training and informative exploration

6.1 The proposer has no admission authority

The reference trains a small tanh network after every executed episode. Features combine the public objective with per-time action frequencies of a candidate program. With normalized observed return y_n, the loss is

$$\ell_n(\theta) = \tfrac{1}{2}\big(f_\theta(g_n, p_n) - y_n\big)^2. \tag{7}$$

One stochastic gradient update uses the selected program and its actual return. There are no evaluator-oracle labels, imaginary rollout labels, or offline optimization stages. A visit bonus encourages underexplored proposals. These features collapse distinct reactive trees and are intentionally weak; the primary library contains only open-loop words.

Every proposal is a typed immutable program. A candidate replaces the incumbent only after an exact check proves strictly positive L_n; ties preserve the incumbent. An optional local-model driver accepts bounded JSON words or trees, not executable code or self-issued certificates. Its parser is tested, but no real LLM service ran in v4. The separate v5 pilot below closes that execution gap without constituting a native benchmark adapter.

Theorem 4 (Proposer-independent retention)**.** *Within a stationary realizable era, suppose every incumbent replacement is an immutable program with a completed check satisfying $L_n \geq 0$. Then for each declared objective g, the true reset values of its successive incumbents are nondecreasing, regardless of how proposals are generated or updated.*

Proof. Theorem 1 retains $m^\star \in \mathcal{C}_n$. Substituting it into the universal bound gives $V_{m^\star}(p, g) \geq V_{m^\star}(b, g)$. Induct over admitted replacements. Proposal quality does not occur in the inequality, only in which candidate is checked. Frozen program semantics and unchanged objective/reset semantics are necessary for the substitution. □

6.2 A non-refundable exploration ledger

For a non-admitted probe p_n, charge before execution

$$d_n = [-L_n(p_n, b_{n,g_n}, g_n)]_+. \tag{8}$$

Only a completed exact comparison can authorize a probe, and cumulative debits must remain at most B. Favorable realized outcomes do not refund debits. Incumbent execution and admitted improvements have zero debit.

Theorem 5 (Completed-prefix deficit)**.** *Under Theorem 4's assumptions, every completed episode prefix N satisfies*

$$\sum_{n=1}^{N} \left[V_{m^\star}(b_{0,g_n}, g_n) - R_n\right] \le \sum_{n=1}^{N} d_n \le B. \qquad (9)$$

Proof. Retention gives $V_{m^\star}(b_{0,g_n}, g_n) \le V_{m^\star}(b_{n,g_n}, g_n)$. Exact debit validity bounds the latter by $R_n + d_n$ for each executed probe; the same relation holds with $d_n = 0$ for incumbent or admitted execution. Summation and the prepaid ledger yield the claim for every completed prefix. □

Equation (9) allows individual losses. It is not an all-state performance guarantee, a guarantee under hidden dynamics changes, or a safety specification for irreversible actions. It applies separately within each contract era, not across unbounded capacity restarts.

6.3 Do not pay for a certainly uninformative probe

Our initial integrated controller could spend repeated debits on programs whose observable outcome was already fixed. We now enumerate a feasible probe's set of distinct observable traces over $\mathcal{C}_n$. A probe is selected only when this set contains more than one trace and $U_n > 0$.

Proposition 6 (Strict evidence reduction)**.** *If a deterministic probe has at least two possible observable traces in $\mathcal{C}_n$, every realized trace removes at least one currently compatible world.*

Proof. Two different possible traces cannot both equal the observed trace. A compatible world realizing the other trace is eliminated by conditioning. □

This is not expected information gain. The controller orders feasible probes by smallest debit, most distinct observable traces, largest optimistic return difference, then proposer rank. Counts are not probabilities and do not count arbitrary latent-state labelings as information. At most $|\mathcal{C}_0| - 1$ strictly eliminating events are possible in a finite class, but this exponential bound does not guarantee sufficient budget, available distinguishing programs, or identification of an optimal policy.

Algorithm 1. One reset episode of the reference learner

1. Rank immutable programs with the online proposer. Compare each candidate with the selected public objective's incumbent using exact paired execution.
2. If a completed check has $L > 0$, promote a candidate with maximal L, breaking ties by proposer order. Execute it without a debit.
3. Otherwise consider only exact checks with $U > 0$ and affordable $[-L]_+$. Enumerate observable traces. If an informative probe is available, select the least debit, then most traces, then largest U, then proposer order. Charge its debit before execution. Otherwise execute the incumbent.
4. Condition the cover on the actual trace. Apply one neural update using only the selected program and observed return. No hypothetical outcome is evidence.

Figure 2: The preserved v4 loop. Version 5 changes only probe selection; certificate admission, immutable execution, evidence updates, and prepaid debits are retained.

7 Inherited v4 experiments, reproduced checks

7.1 Protocol and comparators

Development uses seeds 0–19. An initial seven-arm run contains 6,720 episodes; the revised eight-arm development run contains 7,680. A local protocol record was written before generating 100 fresh seeds, 10000–10099. It is a local freeze, not external preregistration. The primary run comprises eight systems, 48 episodes per seed, and 38,400 episode rows.

Each hidden transducer has two states, two actions, and two outputs. The horizon is four, and all 16 binary action words form the candidate library. The learner does not receive the generated table. Public rewards alternate in three 16-episode blocks: $r_A = (0, 2)$, $r_B = (2, 0)$, then r_A again. The transition model never changes. This is public-objective recurrence, not hidden-regime recurrence. Episode rewards lie in $[0, 8]$.

The strong full-history comparator enumerates all consistent models and selects the word with largest optimistic value, breaking ties by mean model value and then lexicographic order. It has no risk constraint. It is a symbolic planner, *not* raw-history LLM ICL. A last-output model is a weak observation-reactive diagnostic. Other arms are the fixed anchor, Witness at budgets 0, 16, and 64, a no-training ablation, and an unguided-probe ablation.

All systems receive only their own executed feedback and have equal environment horizons. Total CPU time is not matched. Evaluator-only word oracles do not train the agents. The primary metric is mean return over all episodes; late-A return uses the last eight. We additionally track incumbent regressions, risk spent, and maximum completed-prefix deficit against the fixed anchor.

Held-out limitations and audit. The 256-table class is tiny. Development contains 19 distinct tables and the 100 fresh seeds contain 81, with six tables shared be-

System	Mean	Late A	Max. deficit
Fixed anchor	3.9267	3.7800	0
Full-history planner	6.3729	6.4800	2
Last-output model	6.0275	6.1450	52
Witness, $B = 0$	6.2354	6.4000	0
Witness, $B = 16$	6.3358	6.4800	4
Witness, $B = 64$	6.3358	6.4800	4
No neural training, $B = 64$	6.3217	6.4800	4
Unguided probes, $B = 64$	6.2792	6.4800	40

Table 1: Executed primary study: 100 fresh seeds, eight systems, 48 episodes, 38,400 rows. Mean and late-A values are rewards, not percentages. Maximum deficit is the worst completed-prefix anchor deficit across all seeds. The strong full-history planner is symbolic, not an LLM ICL baseline.

tween the two sets. Independent fresh seeds therefore do not establish unseen-world generalization. A secondary new-reward diagnostic in raw JSON preserves objective A's output ordering and is not treated as credible new-domain transfer.

After the first held-out run, exact DAG IDs replaced repeated structural suffix checks and strict candidate registration was added. The controller's selection rule was not tuned on held-out outcomes. The primary run was repeated with the final code. An included audit verifies identical actions/programs, rewards, objectives, deficits, spent risk, and frontier sizes across all 38,400 rows. Original freeze hashes and pre-backend results are retained; timings and search counters are allowed to differ.

7.2 Learning and retention results

Table 1 shows improvement over the fixed anchor, but not over the strong planner. Guided budgets 16 and 64 have mean return 6.3358 versus 6.3729. The paired-seed difference is -0.037083 reward per episode, with a 95% paired t interval of $[-0.055284, -0.018883]$. Average-return superiority is rejected in this experiment. Late-A return ties at 6.48. The planner also has a lower largest observed prefix deficit, 2 versus 4. The conditional bound is not evidence of an empirical risk–return advantage over this planner.

The informative-probe refinement is useful within this controller. Guided B64 exceeds unguided B64 by 0.056667 reward per episode, interval $[0.029805, 0.083528]$, and reduces the largest observed prefix deficit from 40 to 4. Training exceeds the no-training B64 ablation by only 0.014167, interval $[0.007394, 0.020940]$. These secondary comparisons are descriptive and not multiplicity-corrected confirmatory findings. The small neural effect argues against presenting this as a major new training algorithm.

There are zero recorded incumbent regressions under the realizable contract. Mean B16 risk spent is 2.62, versus the allowed 16; mean incumbent promotions are 1.4 per seed. The network receives 48 actual-return updates per seed. Its weight payload is 1,544 bytes, but this excludes features, visits, trees, evidence, frontiers, archives and Python overhead. Raw trace payload grows with episode count. No bounded-total-memory result follows.

7.3 Checker cost and robustness

A separate 60-query CPU diagnostic compares paired execution with and without suffix closure, using seven timing repetitions per query and identical extrema. Median query speedups are 8.41 for identical programs, 1.55 for shared suffixes, and 0.90 for unrelated programs. The identical-program case is a trivial short-circuit, not an end-to-end systems claim. Unrelated programs remain slower. Before DAG IDs, repeated subtree work erased the shared-suffix benefit; those negative measurements are preserved.

A 15-case scaling stress test varies state count and horizon with a 20,000-node cap. Four cases return UNKNOWN. This is correct abstention but unfavorable scaling, not a hidden success. The reference has no new GPU kernel, and its CPU microbenchmarks do not predict GH200 serving performance.

An additional 1,200-episode diagnostic overwrites proposer parameters with large arbitrary values between episodes. Incumbent and budget violations remain zero under the contract. This tests separation of proposal and admission, not adversarial safety of the whole Python process. An unannounced dynamics-change counterexample has old certified gain +2 and actual new-world difference −2. Feedback can detect some contradictions after execution; it cannot protect the first violation of an unobserved assumption.

8 Implementation and proof status

The release adds an exact partial-table learner, paired checker, online agent, strict proposal parser, three experiment drivers, independent C++ rollout reference, and

tests. All 355 inherited Python tests reproduce; the v5 suite adds mechanism, local-pilot, and native-interface checks. Exact small-class tests compare model covers and paired extrema against exhaustive enumeration, including reactive policies, lower-bound witnesses, empty classes, search caps, capacity changes, and wrong-state-bound counterexamples. The neural gradient is checked by finite differences.

The new C++ reference passes 400 paired observation-tree cases and a bounded reward rejection under undefined-behavior sanitization. The preserved legacy suite passes 632 cases plus an overflow check. These tests validate complete numeric rollouts and boundary handling, not a formal refinement of the symbolic search. The inherited v4 release did not run a native benchmark, actual LLM, or CUDA experiment. Version 5 adds actual local-model inference as described below; no new CUDA kernel or native performance result is claimed.

The v5 Lean 4.19.0 audit accepts all 50 inherited declarations and one new history-aliasing obstruction. Two reserved binder names and one Boolean proof step required repair; no inherited hypothesis or conclusion was weakened. An automated inventory prints every theorem's axioms and ties the report to source hashes. Twenty-four declarations are axiom-free; the remainder use only Lean's standard propositional extensionality, quotient soundness, and classical choice. No custom axioms or final `sorryAx` dependencies remain.

This is verification of the encoded abstract statements. For example, a cover splitting lemma does not prove the Python depth-first search implements that split, and a guarded-update lemma assumes a correct bound and true-model membership. Numerical training, search-cap propagation, runtime tickets, C++, and LLM semantics are not formally refined to these assumptions. The source-hashed audit and detailed proof-to-runtime gaps accompany the code.

Proposition 7 (History aliasing prevents uniform strict gain)**.** *Suppose two worlds give a deterministic policy identical available history, including its memory. If every action that strictly improves on an incumbent in the favorable world loses value in the adverse world, a policy preserving the incumbent's value in the adverse world cannot improve strictly in the favorable world at this information state.*

Proof. Identical histories give the same selected action. Strict favorable gain would, by hypothesis, make that action worse than the adverse incumbent, contradicting adverse preservation. The Lean statement instantiates this implication for arbitrary history and action types with integer value functions. □

This elementary obstruction does not forbid safe distinguishing probes, nor claim a new lower bound for randomized or noisy learning. It makes the scope of universal zero-loss discovery explicit.

9 Version 5: a falsified exploration hypothesis

9.1 Decision regret instead of trace count

The inherited controller charges a worst-case loss but orders feasible probes by how many observable traces they distinguish. This can reward distinctions that do not change which program should be executed. We tested a replacement that values uncertainty about the decision itself. For the fixed program library $\mathcal{P}$, define

$$\rho_g(\mathcal{C}) = \min_{p\in\mathcal{P}} \max_{m\in\mathcal{C}} \left[\max_{q\in\mathcal{P}} V_m(q,g) - V_m(p,g)\right]. \tag{10}$$

This is minimax simple regret restricted to the supplied library, not realized cumulative regret or unrestricted optimal control. Model-wise optima are computed inside the learner's compatible class; the true environment is not revealed. Models with identical observable traces for every library program are quotiented before scoring. Label multiplicity is therefore not treated as probability.

Let $\mathcal{C}_{u,\tau}$ be the nonempty cell consistent with observable trace τ after program u. The fixed two-objective score is

$$G(u,\mathcal{C}) = \sum_g \rho_g(\mathcal{C}) - \max_\tau \sum_g \rho_g(\mathcal{C}_{u,\tau}). \tag{11}$$

Among exactly checked, affordable probes, the new controller maximizes $G/(1+d)$, then favors lower debit, larger optimistic current-objective gain, and proposer order. The incumbent is eligible as a free probe. If the best G is zero, the controller executes the incumbent. Strict guaranteed improvements still use the unchanged v4 admission rule. Enumeration has a strict cap and returns UNKNOWN when incomplete. Exactness is conditional, exponential, and limited to the fixed finite model and policy classes.

9.2 Frozen protocol and rejected primary hypothesis

The selection rule was recorded before a 20-seed pilot (20000–20019) and a 100-seed holdout (30000–30099). Every stream uses the inherited two-state, two-action, two-output generator, 16 action words, horizon four, and 48-episode A–B–A objective schedule. Four arms yield 3,840 pilot and 19,200 holdout rows: v4 B16, decision-regret B16, its untrained ablation, and the original optimistic full-history planner. All receive their own feedback and identical interaction horizons. Total CPU time is measured but unequal. The primary contrast is all-episode mean reward of the new trained controller minus v4 B16; intervals pair independent seeds rather than treating episodes as independent.

System	Mean return	Late A	Max. deficit	Seconds/stream
Witness v4, B16	5.9217	5.8000	4	0.0359
Regret probes, B16	5.8271	5.7200	10	0.0680
Regret probes, no training	5.8600	5.8000	10	0.0645
Full-history planner	5.9446	5.8000	4	0.0466

Table 2: New frozen v5 holdout: 100 seeds, 48 episodes, four systems. Rewards range from zero to eight. Deficit is the worst completed-prefix loss relative to the initial anchor. Times cover measured per-stream decision/update mechanics, not setup or a dedicated timing experiment; concurrent host load can affect them.

The proposed improvement is rejected. New mean return is 5.82708 versus 5.92167 for v4: difference -0.09458, paired 95% t interval $[-0.16915, -0.02001]$. Against full-history planning the difference is -0.11750, interval $[-0.19182, -0.04318]$. Late-A return falls from 5.80 to 5.72, with three seeds worse and 97 tied; its paired interval crosses zero. The largest observed anchor deficit increases from 4 to 10, still within the allowed 16. No protected-incumbent violations occur. Mean measured mechanics cost is approximately 1.90 times v4. Fewer paired-check nodes do not imply lower cost: the new arm adds about 5,957 model rollouts per stream for regret scoring.

Training also fails to show a clear benefit in this comparison: the trained minus untrained difference is -0.03292, interval $[-0.09639, 0.03055]$. This secondary interval is descriptive and uncorrected for multiplicity. The result does not contradict conditional incumbent retention; it shows that retention alone neither chooses useful probes nor ensures efficient learning. The pilot contains 20 distinct tables and the holdout 85, with 10 shared. The new holdout also shares seven tables with v4 development and 22 with its holdout. These are samples with replacement from only 256 labelled tables, not disjoint environments or evidence of new-domain transfer.

9.3 Complementary free probes defeat one-step scoring

An independent post-hoc counterexample isolates a completeness failure. There are four compatible worlds indexed by hidden bits (a, b). An anchor returns 5 and reveals nothing. Two other actions also return 5 but reveal a and b, respectively. Two terminal decision actions return 10 or 0 depending on the parity a xor b. Observations encode these outcomes; all four worlds are valid stationary reset models in a separately declared finite class. This construction uses more outputs/actions than the primary binary study and is identified as such in its executable artifact.

Initially the minimax regret is 5. Either free bit probe leaves parity ambiguous, so its immediate regret reduction is zero. The decision actions have positive regret reduction but a worst-case debit of 5 and are infeasible at budget zero. The greedy selector therefore repeats the anchor indefinitely. Nevertheless, executing the two free probes identifies parity and permits a guaranteed improvement of 5 without any exploration loss. The reproducer executes both the stall and the successful probe sequence; it is a finite implementation test and written induction, not an additional Lean theorem.

A separately diagnosed admission plateau. Three frozen-holdout seeds stall after the sum of minimax regrets reaches zero. A common optimal candidate exists, but it ties the incumbent in at least one compatible model, so the strict $L > 0$ promotion rule refuses it. Every probe also has zero regret-reduction score. A separate weak-dominance closure promotes a candidate only when exact comparison gives $L \geq 0$ and $U > 0$, preserving the conditional retention premise. This is a post-hoc repair, not the frozen method.

An exhaustive development diagnostic evaluates all 256 labelled tables under one proposer initialization per table, four arms and 48 episodes: 49,152 rows. The repair improves the original v5 mean by 0.05453, improves ten tables and worsens none, and repairs four late-return deficits in that diagnostic. It still trails v4 and the full-history planner (mean 6.25326 versus 6.26123 and 6.29541) and is slower. This finite schedule census supports the diagnosed repair only; it supplies no held-out confidence interval or guarantee for other schedules, model classes, or random initializations. It does not resolve the complementary free-probe obstruction above.

Successor hypothesis. A bounded, adaptive two-probe plan can value complementary information that Equation (11) misses. For each first outcome, choose a second probe using that branch's updated cover, and maximize the worst-branch terminal reduction. Charge every executed probe separately using the original ledger; simulated outcomes supply planning information, never training evidence. A cheaper baseline simply falls back to v4's trace-count rule when $G = 0$. Neither approach is entitled to a reward guarantee from uncertainty contraction. The first kill test is failure to solve the complementary-probe example at zero budget; the substantive kill test is no future reward advantage over both v4 and the cheaper fallback at equal total compute. The existing holdout is

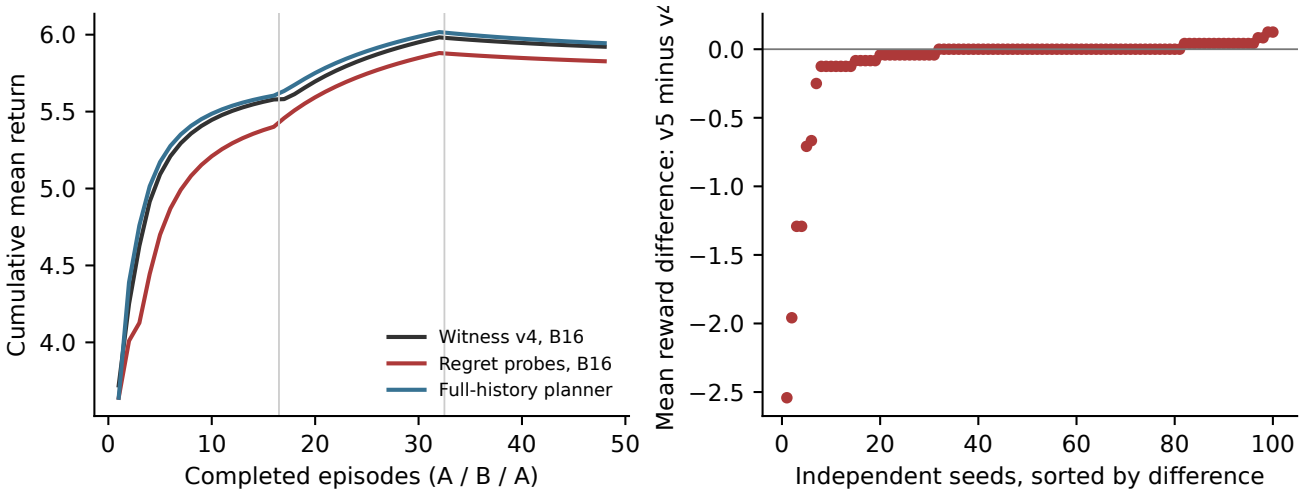

Figure 3: Frozen v5 result from raw episode records. Left: mean cumulative reward, including all exploratory episodes. Right: per-seed mean differences sorted for visibility; the seed is the statistical unit. The public objective changes at 16 and 32 completed episodes. Underlying CSVs accompany the figure.

now consumed for design purposes. Any tuned successor evaluation must be labelled post-hoc or use a genuinely new task family/protocol.

10 Actual local-model inference and native integration

A first actual local-model smoke run used the already available 27B Q3 GGUF model behind a loopback llama.cpp server, with temperature zero, a 96-token output cap, and one call per episode for each model arm. The arms are raw executed-history prompting, the same prompt plus the v4 checker and online proposer, and a checker-based static-library control with no model calls. Programs are frozen before execution; invalid output consumes the call and triggers an explicitly recorded fallback, with the rejection retained. No offline retraining or paid API is used.

The first fixed seed, 41000, produced 36 episodes and 24 model calls with no invalid outputs. All arms returned 5.3333 on average. Evaluator-only enumeration then found every program had identical reward in that world, so this result has no learning headroom. It establishes only executable inference and accounting. A subsequent four-consecutive-seed exploratory extension (41001–41004) produced 144 episodes and 96 model calls. Mean returns were 3.9583 for raw history, 5.5417 for checked history, and 5.0000 for the static-library control. Both model arms used 13,540 input and 576 output tokens, with respectively 141.91 and 128.51 seconds of observed model-call latency. Both stages together contain 120 actual calls, all with valid output and complete usage records. The static-library control still updates the small auxiliary ranker; only its candidate library and absence of LLM calls are static.

These are five environments, not 60 independent trials per arm. No stateless LLM control, native task, or powered retention evaluation ran. The symbolic checker has a strong finite-class prior, more persistent state, extra CPU work, and online auxiliary training. No LLM weights change. The pilot supports integration feasibility and motivates a native experiment; it does not isolate memory quality, parameter learning, or cost-matched superiority. Initial model loading took about 77.7 seconds and is excluded from call timings. Host/GPU memory values are environment observations, not isolated peak-memory benchmarks.

An independent review found misleading malformed-output metadata and incomplete usage-field handling in the pilot driver. Neither branch was exercised in these runs: every completion was valid and every usage record complete. Future-run code and regression tests are corrected; the exact executed source, original results, and review are preserved. For invalid future output, raw history uses the anchor while the checked arm skips registration and uses its existing controller, with no extra call. This distinction is now explicit.

The native integration targets the official CL-Bench interface at commit `5f8c50eb1e84b2eda2ef4faff757dfc812a0ea26`. Its feedback semantics are material: the runner separately delivers observations; duplicating them through query handling would change the ICL baseline. A task may legally reveal a correct answer in post-action feedback; that is different from reading the evaluator's answer records before acting. Native reward and each system's stateful–stateless gain must remain separate. A bridge and interface tests do not supply a native benchmark result, an empirical retention result, or a valid finite-state abstraction for database drift. Those remain the next experimental obligations.

Twenty optional-runtime tests pass, including eight tests executing the actual pinned upstream ICL/runner and SQLite task; twelve dependency-free contracts also belong to the 401-test standard suite. The native transport is scripted for these checks, and the SQLite database/questions are fixtures. Its exact certificate metadata is always UNKNOWN. The bridge is a baseline and audit foundation, not an implemented native Witness adaptation algorithm. The unmodified official benchmark dataset and a live native model transport remain unexecuted.

11 Bounded nonmyopic probing and its limits

Version 6 asks whether two-observation lookahead repairs the greedy plateau at useful cost. This phase uses finite CPU simulators with every neural ranker frozen; visit counters and exact evidence still update after actual episodes. No new language-model inference, weight adaptation, deployment, or scheduled continuation is part of this phase.

Established antecedents. Observation-contingent policy trees and belief-state planning are established POMDP methods [14]. POMCP and Bayes-adaptive sampled search already value information through future decisions while avoiding full tree expansion [15, 16]. Their sampled objectives do not certify a worst-case bound over every compatible model. Adaptive submodularity provides conditions for effective greedy acquisition; the parity counterexample violates the relevant diminishing-returns intuition [17]. EC^2 instead cuts edges between hypotheses that require different decisions, so useful tests need not immediately improve attainable reward [18]. Overlapping acceptable decision regions require additional care, as in hyperedge cutting [19]. Neither lookahead nor decision-directed acquisition is a novelty claim here.

Algorithm and retained authority. All three v6 selectors first attempt an exact strict or weak improvement using the unchanged paired checker. When that fails, the depth-two selector enumerates the compatible models and plans a first probe with a different second probe allowed for each observed trace. For the library regret $R(C)$ defined in the v5 study, a contingent plan $\pi = (p, \{q_\tau\})$ has score

$$S(\pi) = \frac{R(C) - \max_{\tau,\sigma} R(C_{p,\tau,q_\tau,\sigma})}{1 + d_p + \max_\tau d_{q_\tau}}.$$

Stopping is allowed in every branch; all debits use exact comparisons with the current incumbent and every branch must fit the remaining budget. The current incumbent and goal remain fixed within this hypothetical plan. For each first probe, we enumerate attainable terminal-regret thresholds and choose the cheapest feasible second alternative in each branch under each threshold. This finite procedure covers the optimum of the stated gain/debit surrogate. Independently minimizing regret in each branch can waste debit below the worst branch's bottleneck; a counterexample and regression tests reject that earlier variant. The whole selector is still restricted by its promotion shortcut, program library, proposal set, and computation caps.

Only the first action program executes, after a final exact comparison. The second-probe map is an explanation, not a future execution ticket. Real feedback causes replanning, so later goal changes or computation limits can prevent the predicted two-step gain from materializing. Simulated traces never enter the experience record. A plan that exhausts its computation allowance returns UNKNOWN, keeps the incumbent, and spends zero additional debit.

The cheap control chooses an informative program with a nonnegative exact lower return difference, including reward ties. It uses possible-trace search without enumerating complete models. A second control maximizes optimistic current return under the same exact checks and B16 ledger. All three have the same finite inputs and a common planning ceiling: two million package Python line events, a cooperative one-second deadline, 4,096 distinct models, and 100,000 nodes per paired check. Counters include attempted duplicate completions and interrupted search work. They exclude their own bookkeeping, construction, observation updates, and C-backed/library internals. This is neither a FLOP metric nor an operating-system resource sandbox. Actual elapsed costs are reported separately.

Positive and negative controls. Every selector obtains returns $(5, 5, 10)$ on all four two-bit parity worlds at B0. Thus two-step planning repairs the motivating failure, but deeper planning is unnecessary for that repair. In the analogous three-bit class, every B0-feasible plan of at most two probes has zero regret reduction. The depth-two rule remains at return five in all eight worlds, while the cheap rule obtains $(5, 5, 5, 10)$. The same construction defeats every fixed planning depth at B0 when it contains more hidden bits than the lookahead. Equal-payoff controls also charge extra planning without any possible reward gain. These are declared development mechanisms, not independent benchmark validation.

Frozen held-out protocol. The primary family contains all 1,296 labelled two-state, two-action, three-output transducers. The library has eight length-three action words, and the public objectives have rewards $(0, 1, 2)$ and $(2, 1, 0)$. Each 24-episode stream uses eight episodes of A, eight of B, then eight of A; future schedule

Method	Return	Plan s	Unknown	Max debt
Depth two	4.197	8.344	30.7%	0
Free info.	3.803	0.940	0.0%	0
Bounded opt.	4.392	1.560	0.1%	9
V4 frozen	4.391	0.010	—	6
Full history	4.432	0.036	—	6

Table 3: Frozen v6 holdout: mean episode return, total planning seconds per 24-episode stream, and maximum completed-prefix anchor deficit. Only the first three arms share line metering and the one-second ceiling. Full-history optimism has no risk ledger. All ranker weights stay frozen. Timing includes measurement overhead.

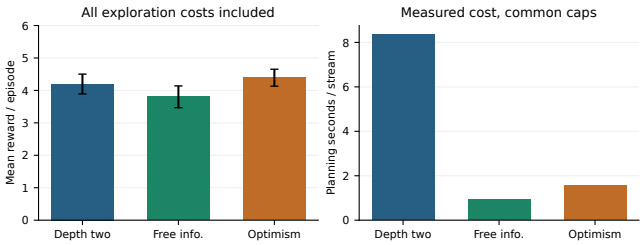

Figure 4: Reward and measured planning cost for the three commonly metered v6 arms. Reward intervals are marginal 95% Student intervals across streams; the primary analysis uses paired differences. Common ceilings do not imply equal realized computation.

and true world tables never enter the learner. Development uses seeds 60000–60007; 40 held-out streams use seeds 61000–61039 after a local source/configuration freeze. Seeds sample tables with replacement. Correcting the meter's path handling and the branch-debit scoring defect occurred before this freeze; the initial and corrected development outputs are both retained. This is local protocol recording, not external preregistration.

The primary mean difference is +0.39375 reward per episode, with paired 95% Student interval $[+0.15845, +0.62905]$ over 40 independent streams. The depth-two selector uses 8.88× the cheap control's measured planning time. The holdout contains 40 distinct tables; 0 streams match a development table (0 distinct overlapping tables). This is same-family validation, not new-domain generalization.

The positive primary contrast does not establish efficiency superiority. Relative to bounded optimism, depth two loses 0.19479 reward per episode (paired 95% interval $[-0.31761, -0.07198]$); it also loses 0.19375 to the frozen-ranker v4 control and 0.23542 to unbounded full-history optimism. These are secondary, unadjusted contrasts. Mean charged risk is 1.775 for depth two, zero for the free control, and 6.975 for bounded optimism. The common budget is an upper ceiling, so the positive contrast does not isolate lookahead from willingness to spend risk. No guarded class, incumbent, or budget audit fails; all 3,840 guarded holdout prefixes satisfy B16.

Depth two returns UNKNOWN on 30.7% of decisions. The efficiency-superiority hypothesis is unsupported, while the narrowly stated reward contrast against the zero-loss rule is supported on this family.

An executable formal bridge. The Lean inventory now has 63 kernel-checked declarations, including 12 new statements about an executable finite transducer interpreter and history filter. Let $\mathrm{run}(M, w)$ be the observable trace from reset and let $H_M(W)$ be the history generated by actually running each word in W. The checked characterization is

$$\begin{aligned} m \in \mathrm{filterHistory}(C, H) \\ \iff m \in C \ \wedge\ \forall t \in H, \\ \mathrm{run}(m, \mathrm{actions}(t)) = t. \end{aligned}$$

It follows constructively that $M \in C$ implies $M \in \mathrm{filterHistory}(C, H_M(W))$. This relates evidence retention to execution rather than assuming an uninterpreted consistency predicate. All 63 declarations use only the stated standard Lean dependencies and no placeholders or custom axioms; 24 are axiom-free.

An independently compiled Lean executable supplies 7,936 traces and 1,024 history survivor sets over all 256 binary two-state machines. Python matches all traces, all 7,680 nonempty word-program executions, and 262,144 filter membership decisions. These are finite differential checks of the Python bridge, not a general refinement proof of the partial-table DFS or adaptive policy checker. The theorem still requires genuine feedback, reliable reset, stationary deterministic dynamics, and initial inclusion of the true machine. It establishes neither alignment nor a native continual-learning win.

12 Limitations and decisive next experiments

Where the guarantee breaks. Consistency with observed traces does not validate K, stationarity, reset semantics, deterministic feedback, or output sufficiency. A point estimate or underspecified class can certify a change that fails in a larger compatible world. Hidden drift can invalidate a previous certificate before its first counterexample arrives. Public objectives do not solve hidden task routing. The fixed reset-value theorem is weaker than v3's supplied-model all-state claim and much weaker than broad capability retention.

There is also an unavoidable information–risk tradeoff within some classes. Two worlds can make the anchor observationally identical while a distinguishing action has opposite reward consequences. Zero-risk execution cannot both retain the anchor in the unfavorable world and guarantee learning the favorable one. The right target is therefore not an unconditional monotonicity slogan, but a useful, explicit and affordable tradeoff.

Independent new information can also require growing total storage; our archive does not bypass that cost.

Native falsification. The next experiment must first acquire a valid abstraction from a real, resettable tool environment without using hidden tables or task answers. Then evaluate compatible CL-Bench domains and AgentCL streams with the same fixed backbone, permissions, generation settings and total compute budget. Include raw-history ICL, a strong memory/skill baseline, an evolving harness, point-estimate models, and removals of checking, training and informative probes. Count proposal sampling, gradient updates, checker work, retrieval, replay, resets, retries and environment interactions. An oracle-representation arm is diagnostic only.

A concrete proposed success criterion is at least five percentage points of native normalized learning gain over the strongest cost-matched baseline, positive lower paired 95% bound, and no protected old-task success loss larger than two percentage points. These are research targets, not achieved results. Reject the scalable hypothesis if no sufficient small abstraction survives held-out continuation tests, if over 90% of candidate checks return UNKNOWN on the selected native tasks, if apparent gains vanish at matched compute, or if cost exceeds the strongest baseline without the predeclared gain.

Resources and execution design. The current simulator requires CPU only. The first local inference pilot uses an existing 27B Q3 model with 48 GPU-offloaded layers on an RTX 5070 Ti, 4,096 context tokens and a single server slot. A larger native run needs a measured fixed-backbone inference budget and, where required, a Docker host. Measure native pilot throughput before assigning GPU-hours; the small synthetic run does not establish native task throughput. Retain full provenance and measure actual peak host/GPU memory rather than payload estimates.

A possible systems extension groups partial-transition work by time and unresolved slot in structure-of-arrays queues, with exact residual IDs and checked reductions. CPU parallelism should precede a GPU rewrite for tiny irregular checks. A proof cache must include immutable programs, objective, reset contract and evidence ancestry; enlarging the class invalidates reuse. These are unimplemented engineering hypotheses. A GPU cannot remove exponential model ambiguity, and an approximation cannot inherit exact guarantees without a separate soundness argument.

13 Conclusion

Witness-CL v0.6 learns compatible hidden-state models from its own executed traces and uses them to protect a behavioral incumbent independently of a mutable proposer. Version 5 adds checked abstract Lean statements, real local inference, and a falsified regret-probe heuristic. Version 6 executes the bounded successor, exposes a stronger three-bit obstruction, and links evidence retention to an executable Lean interpreter. The v6 primary gain over free information is positive, yet bounded optimism still wins at much lower measured planning cost within the same B16 ceiling. Actual risk spending differs. These results separate retention, information acquisition, and learning efficiency. It does not yet supply the requested general stateful benchmark winner. The remaining research obligation is concrete: learn sufficient continuation abstractions cheaply enough that verified reuse beats full-history learning at equal cost, without hiding class misspecification or old-skill loss behind the word "memory."

Reproducibility and disclosure

Source, tests, raw episode files, result generation, original Git history, initial and successful Lean build evidence and paper sources are included in the private research repository. The main reproduction targets are `make test`, `make cpp-check`, `make latent-heldout`, `make latent-diagnostics`, and `make paper`. The v5 local-model pilot was executed; its small scope is explicit. This is an AI-assisted research draft prepared for Samuel Mausberg, with author review and independent replication pending. The native learning problem remains open.

References

[1] Parth Asawa, Christopher M. Glaze, Gabriel Orlanski, Ramya Ramakrishnan, Benji Xu, Asim Biswal, Vincent Sunn Chen, Frederic Sala, Matei Zaharia, Joseph E. Gonzalez. Continual Learning Bench: Evaluating Frontier AI Systems in Real-World Stateful Environments. 2026. https://arxiv.org/abs/2606.05661.

[2] Yiheng Shu, Bernal Jiménez Gutiérrez, Saisri Padmaja Jonnalagedda, Yuguang Yao, Huan Sun, Yu Su. AgentCL: Toward Rigorous Evaluation of Continual Learning in Language Agents. 2026. https://arxiv.org/abs/2606.02461.

[3] Qizheng Zhang, et al. Agentic Context Engineering: Evolving Contexts for Self-Improving Language Models. 2025. https://arxiv.org/abs/2510.04618.

[4] Yufei He, et al. EvoTest: Evolutionary Test-Time Learning for Self-Improving Agentic Systems. 2025. https://arxiv.org/abs/2510.13220.

[5] Jun Nie, et al. TTHE: Test-Time Harness Evolution. 2026. https://arxiv.org/abs/2607.08124.

[6] Yiming Xiong, Shengran Hu, Jeff Clune. Learning to Continually Learn via Meta-learning Agentic Memory Designs. 2026. https://arxiv.org/abs/2602.07755.

[7] Andrei A. Rusu, et al. Progressive Neural Networks. 2016. https://arxiv.org/abs/1606.04671.

[8] Florent Delgrange, Raphael Avalos, Willem Röpke. Deep SPI: Safe Policy Improvement via World Models. In *International Conference on Learning Representations*, 2026. https://arxiv.org/abs/2510.12312.

[9] Donghao Li, Ruiquan Huang, Cong Shen, Jing Yang. Near-optimal Conservative Exploration in Reinforcement Learning under Episode-wise Constraints. In *International Conference on Machine Learning*, 2023. https://arxiv.org/abs/2306.06265.

[10] Satinder Singh, Michael James, Matthew Rudary. Predictive State Representations: A New Theory for Modeling Dynamical Systems. In *Proceedings of the Twentieth Conference on Uncertainty in Artificial Intelligence*, 2004. https://arxiv.org/abs/1207.4167.

[11] Masaki Waga, Ezequiel Castellano, Sasinee Pruekprasert, Stefan Klikovits, Toru Takisaka, Ichiro Hasuo. Dynamic Shielding for Reinforcement Learning in Black-Box Environments. In *Automated Technology for Verification and Analysis*, 2022. https://arxiv.org/abs/2207.13446.

[12] Weijia Xu, Alessandro Sordoni, Chandan Singh, Zelalem Gero, Michel Galley, Xingdi Yuan, Jianfeng Gao. Test-Time Learning with an Evolving Library. 2026. Version 2, July 14. https://arxiv.org/abs/2605.14477.

[13] Longtao Hu, Xiao Liang, Linchao Zhu. From Interaction Traces to Persistent Skills: Online Evolution for Computer-Use Agents. 2026. Submitted September 4. https://arxiv.org/abs/2609.04869.

[14] Leslie Pack Kaelbling, Michael L. Littman, Anthony R. Cassandra. Planning and acting in partially observable stochastic domains. *Artificial Intelligence*, 101:99–134, 1998. https://www.cassandra.org/arc/papers/aij98.pdf.

[15] David Silver, Joel Veness. Monte-Carlo Planning in Large POMDPs. In *NeurIPS*, 2010. https://papers.nips.cc/paper_files/paper/2010/file/edfbe1afcf9246bb0d40eb4d8027d90f-Paper.pdf.

[16] Arthur Guez, David Silver, Peter Dayan. Efficient Bayes-Adaptive Reinforcement Learning using Sample-Based Search. In *NeurIPS*, 2012. https://arxiv.org/abs/1205.3109.

[17] Daniel Golovin, Andreas Krause. Adaptive Submodularity: Theory and Applications in Active Learning and Stochastic Optimization. *Journal of Artificial Intelligence Research*, 42:427–486, 2011. The linked v5 manuscript includes the 2017 correction. https://arxiv.org/abs/1003.3967v5.

[18] Daniel Golovin, Andreas Krause, Debajyoti Ray. Near-Optimal Bayesian Active Learning with Noisy Observations. In *NeurIPS*, 2010. https://papers.nips.cc/paper_files/paper/2010/file/1e6e0a04d20f50967c64dac2d639a577-Paper.pdf.

[19] Shervin Javdani, Yuxin Chen, Amin Karbasi, Andreas Krause, J. Andrew Bagnell. Near Optimal Bayesian Active Learning for Decision Making. In *AISTATS*, 2014. https://arxiv.org/abs/1402.5886.